

Challenge — FairBet Lab

Simulador de apuestas deportivas con moneda virtual

Contexto

Construir una plataforma web educativa en la que los usuarios apuesten **fichas virtuales** (sin valor monetario real) sobre eventos deportivos, con énfasis en **la integridad financiera, el tiempo real, el juego responsable y el cumplimiento normativo** conforme a la Ley 31557 y su reglamento DS 005-2023-MINCETUR.

Restricción explícita del reto: el sistema no integra pasarelas de pago reales, no convierte fichas a dinero, y debe mostrar en el footer: «Plataforma educativa con moneda virtual. No constituye una casa de apuestas.»

Objetivos de aprendizaje

- Diseño de un **wallet con contabilidad de partida doble** (cada movimiento = débito + crédito, saldo derivado, nunca almacenado).
- Manejo riguroso de **decimales y dinero** (nada de *float*).
- **Máquinas de estado** del ciclo de vida de una apuesta.
- Concurrencia: bloqueo pesimista (*select_for_update*), idempotencia y prevención de doble gasto.
- Tiempo real para cotizaciones (odds) y eventos en vivo.
- Implementación de controles de **juego responsable** como requisito funcional.
- Auditoría inmutable y trazabilidad regulatoria.

Alcance funcional

Nivel 1 — Núcleo obligatorio (55%)

1. Registro y KYC (Conoce tu cliente) simulado

- Validación de mayoría de edad (≥ 18) con fecha de nacimiento + DNI peruano (algoritmo de dígito verificador).
- Estados de cuenta: *pendiente_verificacion*, *verificado*, *bloqueado*, *autoexcluido*.

2. Wallet con partida doble

- Modelo *LedgerEntry* con *account*, *amount*, *direction* (DEBIT/CREDIT), *transaction_id*.
- Toda operación financiera crea como **mínimo dos entradas balanceadas** (la suma debe ser cero).
- Cuentas: *wallet_usuario*, *casa*, *apuestas_pendientes*, *bonos*.
- Operaciones: recarga simulada de fichas, retiro simulado, transferencia interna.
- Saldo siempre se calcula por $SUM(credits) - SUM(debits)$, nunca se guarda.

3. Catálogo de eventos y mercados

- Eventos deportivos con estados *programado*, *en_vivo*, *finalizado*, *suspendido*, *anulado*.
- Mercados mínimos: **1X2** (gana local, empate, gana visitante).
- Cuotas (odds) en formato decimal europeo (ej. 2.50), con margen del operador configurable.

4. Apuesta simple

- Usuario elige mercado + selección + monto.
- Validaciones: saldo suficiente, cuenta verificada y no autoexcluida, evento aún no iniciado, monto dentro de límites min/max.
- Al confirmar: bloqueo de fondos vía partida doble (wallet → apuestas_pendientes), creación de *Bet* en estado *accepted*.
- **Liquidación:** cuando admin/sistema marca resultado, se calcula automáticamente $payout = stake \times odds$ y se acreditan ganancias o se libera la pérdida a la casa.

5. Controles de juego responsable (obligatorios)

- Límite diario, semanal y mensual de “depósito” de fichas, configurable por el usuario (solo se puede **bajar** instantáneamente; **subirlo** requiere cooldown de 24h).
- Autoexclusión: temporal (7/30/90 días) o indefinida; el usuario no puede revertirla antes del tiempo.
- Mensaje obligatorio de “consumo responsable” visible en todas las pantallas de apuesta.

Nivel 2 — Avanzado (25%)

6. Apuestas combinadas (acumuladoras)

- Múltiples selecciones; cuota final = producto de cuotas individuales.
- Si **una** falla, la combinada pierde.
- Validación de selecciones mutuamente excluyentes (no se puede combinar “gana local” y “gana visitante” del mismo partido).

7. Cuotas en tiempo real

- Django Channels: canal por evento, actualización de odds en vivo.
- Política de re-cotización: si la cuota cambió entre que el usuario abrió el ticket y confirmó, mostrar nueva cuota y exigir reconfirmación.

8. Apuestas en vivo (in-play)

- Apuestas mientras el evento está *en_vivo*, con cuotas dinámicas.
- Suspensión automática de mercado en eventos críticos (gol, expulsión) por N segundos.

9. Cash-out

- El usuario puede cerrar anticipadamente una apuesta *accepted* a una cuota de cash-out calculada.
- Fórmula sugerida: $cashout = stake \times odds_original / odds_actual \times factor_casa$.

10. Mercados adicionales

- Over/Under 2.5 goles, ambos equipos anotan (BTTS), handicap asiático simple.

Nivel 3 — Compliance, auditoría y operación (20%)

11. Auditoría inmutable

- Tabla append-only de auditoría con encadenamiento por hash ($hash_n = SHA256(hash_n-1 + payload_n)$).
- Cada bet, cada movimiento de wallet, cada cambio de odds queda registrado.

- Endpoint admin de verificación de integridad de la cadena.

12. Anti-fraude básico

- Reglas: misma IP con N cuentas distintas, patrones de apuestas idénticas en grupo, depósitos inmediatos seguidos de cash-out, etc.
- Sistema de alertas (*SuspiciousActivity*) para revisión manual del admin.

13. Dashboard del operador

- Métricas en vivo: GGR (Gross Gaming Revenue = stakes – payouts), exposure por evento (cuánto pierde la casa si gana cada selección), volumen de apuestas, número de usuarios activos.
- Reporte mensual exportable estilo MINCETUR (formato CSV con columnas de la normativa).

14. Bonos promocionales (opcional, +3 pts)

- Bono de bienvenida con rollover (debe apostarse N veces antes de poder retirar).
- Detección de abuso de bono (apuestas sin riesgo: muy alta cuota vs muy baja cuota cubriendo todos los resultados).

Requerimientos técnicos

Stack obligatorio

- Django 5.x + DRF, PostgreSQL, Redis, Celery, Django Channels.
- *Decimal* con precisión configurada (*max_digits=18*, *decimal_places=4*); prohibido usar *float* en montos.
- Transacciones atómicas con *select_for_update* en todo movimiento de wallet.
- Idempotency keys en endpoints de apuesta y de movimiento de fondos.

Calidad de código

- Cobertura mínima **80%** en apps *wallet* y *betting* (estas son críticas).
- Property-based testing con *hypothesis* para las invariantes financieras:
 - “La suma global de débitos y créditos siempre es cero.”
 - “Ningún wallet termina con saldo negativo.”
 - “El payout de una apuesta ganadora siempre es *stake* × *odds* con precisión exacta.”
- Pruebas de concurrencia: simular N peticiones simultáneas y verificar que no haya doble gasto.

Seguridad

- Rate limiting agresivo en endpoints de apuesta y autenticación.
- 2FA opcional para retiros simulados.
- Logs estructurados (JSON) sin información sensible.

Entregables — producto

- Repositorio Git con README, *docker-compose*, seed de eventos y usuarios.
- Diagrama ER + diagrama de la **máquina de estados de Bet**.
- Documentación OpenAPI.
- Reporte de cobertura.
- Documento corto (2–3 páginas) explicando: cómo el diseño garantiza integridad financiera; qué decisiones de juego responsable se tomaron; qué requisitos de la Ley 31557 quedan cubiertos y cuáles no (autocrítica honesta).

- Video demo (7–10 min).

Entregables — proceso (evidencia de autoría)

Dado que este reto involucra integridad financiera y concurrencia, **el rigor de proceso es parte de la evaluación**, no un extra. La sección final del documento, *Política de evaluación y autoría*, detalla plantillas y puntos de control.

- Carpeta `/docs/adr/` con **mínimo 10 ADRs**. Obligatorio al menos uno sobre: modelo de partida doble, manejo de Decimal y precisión, estrategia de concurrencia (*select_for_update* vs optimista), idempotencia en endpoints de apuesta, máquina de estados de Bet, política de re-cotización, controles de juego responsable y auditoría inmutable.
- Carpeta `/docs/sketches/` con bocetos a mano: ER del wallet, máquina de estados completa de *Bet*, secuencias “apuesta → liquidación” y “cash-out”.
- Historial de commits con **Conventional Commits**. Para las apps *wallet* y *betting* se exige **TDD evidente**: commits *test*: antes de *feat*: para la lógica crítica.
- Bitácora semanal individual.
- Documento `/docs/lecciones.md` con **mínimo 4 intentos fallidos** por sprint (dada la complejidad del tema).
- Declaración de uso de IA en `/docs/anti-ai-disclosure.md` por cada integrante. La honestidad valora positivamente; el ocultamiento detectado en walkthrough penaliza fuertemente.

Rúbrica de evaluación (sobre 20)

Criterio	Pts
Wallet con partida doble + invariantes verificadas con tests	4
Máquina de estados de apuesta + liquidación correcta	3
Manejo de concurrencia (sin doble gasto demostrable)	2
Cuotas y apuestas en tiempo real funcionales	2
Controles de juego responsable completos y bloqueantes	2
Auditoría inmutable + dashboard operador	2
Combinadas, in-play, cash-out (al menos 2 de 3)	2
Calidad de pruebas (cobertura + property-based)	2
Documento de compliance + video demo	1
Total	20

Política de evaluación y autoría

Común a ambos challenges

¿Por qué este apartado?

Estos challenges se evalúan en un contexto en el que las herramientas de IA generativa están al alcance de todos. El objetivo del curso no es entregar software que funcione: es que **cada estudiante haya pensado, decidido y construido lo suficiente como para entender lo que entregó**, mantenerlo, mejorarlo y defenderlo. Esta sección describe los mecanismos de verificación de la autoría y los espacios en los que el estudiante debe ejercer su propio criterio.

1. Política de uso de herramientas de IA

La IA es bienvenida como **apoyo al aprendizaje**: explicar conceptos, comparar alternativas, ayudar a depurar entendiendo la causa raíz. La IA **no es** un sustituto del estudiante como autor del código entregado.

Permitido:

- Pedir a la IA que explique un concepto, un mensaje de error o un fragmento de documentación.
- Comparar enfoques (ej. “explícame trade-offs entre WebSockets y SSE para mi caso”).
- Pedir revisión crítica de tu propio código.
- Generar boilerplate trivial (configuración inicial, scaffolding) declarándolo.

No permitido:

- Copiar y pegar bloques generados por IA al repositorio sin entenderlos y poder defenderlos línea por línea.
- Encargar a la IA la implementación de la lógica de negocio crítica (cálculo de la tabla, wallet, máquina de estados, anti-fraude).
- Ocultar el uso de IA en el archivo de declaración.

Obligatorio:

- Cada integrante mantiene su propia `/docs/anti-ai-disclosure.md` indicando qué partes consultó con IA y para qué (estudiar concepto / depurar error / revisar código / generar boilerplate).
- Cuando un commit contiene asistencia significativa de IA, marcarlo en el mensaje con el sufijo `[ai-assisted]`.

Prueba de autoría: cualquier sección del código que el estudiante no pueda explicar y modificar en vivo durante el walkthrough se considera no entregada para fines de su nota individual, sin importar que el código funcione.

2. Plantilla mínima de ADR

Cada ADR es un archivo numerado en `*/docs/adr/*` (ej. `0007-uso-de-channels-vs-sse.md`) con esta estructura mínima:

- **Contexto:** ¿qué problema o decisión enfrentamos?
- **Opciones consideradas:** al menos 2, con pros y contras concretos.
- **Decisión:** qué se eligió.

- **Consecuencias:** qué se vuelve más fácil, qué se vuelve más difícil, qué deudas técnicas se asumen.
- **Fecha y autor.**

3. Reglas de commits

- **Conventional Commits:** *feat.*, *fix.*, *refactor.*, *test.*, *docs.*, *chore.*, *perf.*
- **Atomicidad:** un cambio lógico por cada commit. Un commit de “implementación completa” con 1500 LOC es señal de alarma y activa la revisión.
- **Distribución temporal:** commits a lo largo del periodo, no concentrados en los últimos 3 días previos a la entrega.
- **Co-autoría:** Indicar el nro de grupo y sus integrantes
- **Marca [ai-assisted]** cuando el commit contenga asistencia significativa de IA.

4. Defensa oral final

20” minutos por equipo al cierre del challenge. Preguntas conceptuales sobre las decisiones tomadas; cada integrante responde individualmente al menos una pregunta. Ejemplos:

- ¿Por qué eligieron este stack de tiempo real y no otro?
- ¿Qué pasa si mañana la carga se multiplica por 10?
- ¿Cuáles son las debilidades conocidas de su diseño y cómo las mitigarían en una v2?
- (C2) ¿Cómo justifican que el saldo siempre será correcto, aunque el servidor se reinicie en medio de una transacción?

5. Criterios de individualización (anticopia entre equipos)

Para evitar que un equipo entregue la solución de otro, cada equipo recibe variaciones específicas que no aparecen en este documento público:

Variación funcional única por equipo (entregada en privado al inicio del challenge):

- (C1) “Tu reto incluye, además, el desempate por fair play (cantidad de tarjetas amarillas y rojas) con su prueba dedicada.”
- (C1) “Tu reto requiere mantener estadísticas históricas: cuántos partidos jugó cada selección en mundiales anteriores (data mock provista).”
- (C2) “Tu reto requiere soportar el mercado ‘goleador exacto’ con manejo de empate técnico.”
- (C2) “Tu reto requiere implementar el bono de bienvenida con rollover (que el enunciado deja opcional).”

Decisiones intencionalmente ambiguas (cada equipo decide y documenta en un ADR):

- (C1) ¿Cómo manejar partidos suspendidos a efectos de la tabla? ¿Y si se reanudan en otra fecha?
- (C1) ¿Permiten predicciones después del kick-off con penalización, o cierran al pitazo inicial?
- (C2) ¿Cuántos eventos críticos seguidos suspenden el mercado in-play, y por cuánto tiempo?
- (C2) ¿Cómo definen el umbral del anti-fraude para minimizar falsos positivos sin perder cobertura?

6. Cálculo de la nota individual

Reconociendo que el trabajo es en equipo pero que la nota debe reflejar el aporte personal:

$$\text{Nota_individual} = \text{Nota_equipo} \times \text{Factor_defensa}$$

Donde Factor_defensa se obtiene del walkthrough, hot fix y defensa oral:

- **1.00** — Defiende todo su código con fluidez, explica las decisiones y propone alternativas.
- **0.85** — Defiende su código con dudas menores; falla en algún caso límite.
- **0.70** — Conoce el *qué*, pero no el *por qué* en varias secciones.
- **0.50** — Vacíos significativos; no puede explicar partes que dice haber escrito.
- **0.25** — Solo conoce funcionalidades a nivel demo, no a nivel implementación.
- **0.00** — No puede defender ninguna sección del código atribuido (se asume no autoría).